

Enhancing Subjective Q&A Labeling with Hierarchical ModernBERT and Distribution-Aware Post-Processing

Shaochieh Huang* An Li* Weifan Ching* Yuchen Li*

ABSTRACT

The Google QUEST Q&A Labeling task requires assessing the subjective quality of question-answering pairs across 30 diverse attributes, extending beyond traditional binary classification to fine-grained regression. In this work, we propose a robust framework based on the **ModernBERT** architecture. To capture hierarchical linguistic features, we implement **Weighted Layer Pooling**, dynamically aggregating information from deeper transformer layers. We further enhance feature representation through a **Multi-Pooling Strategy** (combining [CLS], Mean, and Max pooling) and improve generalization using **Multi-Sample Dropout**. The model is trained with a **Hybrid Loss Function** that combines Binary Cross Entropy and a differentiable Spearman Loss to directly optimize the competition metric. Finally, we introduce a post-processing **Rank Optimization** technique (Quantile Snapping) to align continuous model predictions with the discrete distribution of human annotations. Our approach demonstrates that combining advanced pooling mechanisms with distribution-aware post-processing significantly improves the Spearman rank correlation.

I. TASK DESCRIPTION

1.1 Objective

The primary objective of the Google QUEST Q&A Labeling task is to develop a predictive model that can automatically assign subjective scores to question-answer (Q&A) pairs. Unlike traditional Natural Language Processing (NLP) tasks that focus on binary classification (e.g., spam detection) or extractive question answering, this task requires the quantification of human-centric attributes. The system must process three primary textual inputs: *question_title*, *question_body*, and *answer*, and

output a continuous vector representing 30 distinct subjective properties.

1.2 Dataset and Target Labels

The dataset originates from the Stack Exchange network, comprising diverse topics ranging from technical programming to life skills and hobbies. The 30 target labels are normalized between 0 and 1 and are categorized into two groups:

- **Question-based Labels (21 targets):** These evaluate the nature and quality of the question itself, including metrics such as *question_asker_intent_understanding*, *question_well_written*, and *question_opinion_seeking*.
- **Answer-based Labels (9 targets):** These evaluate the quality and relevance of the response in relation to the question, including *answer_relevance*, *answer_satisfaction*, and *answer_type_instructions*.

1.3 Evaluation Metric

The performance of the model is evaluated based on the mean Spearman's rank correlation coefficient. For each of the 30 target columns, the correlation between the predicted ranking and the ground-truth ranking is calculated. The final score is the average of these 30 correlation values. This metric emphasizes the model's capability to maintain the relative order of subjective intensity rather than minimizing absolute numerical error.

1.4 Technical Challenges

- **Contextual Understanding:** Distinguishing between "fact-seeking" and "opinion-seeking" questions requires deep semantic comprehension.
- **Noise in Crowdsourcing:** Since labels are generated by multiple human annotators,

the model must be robust against the inherent variance in subjective human judgment.

- **Multi-output Regression:** Optimizing a single model to perform well across 30 different, yet potentially correlated, tasks necessitates an effective multi-task learning strategy.

II. RELATED WORKS

2.1 BERT-based Fine-tuning Approach by Khanh

This research draws heavily from the methodology proposed by Khanh, which utilizes a BERT-base-uncased model to handle the multi-label task. The core of this approach lies in the input structure. Three text fields—title, body, and answer—are joined into a single sequence using the [SEP] token. This allows the model to learn the link between the user's question and the given response. To solve the regression problem, a linear layer is added on top of the BERT encoder. This layer outputs a vector of 30 values, which are then passed through a Sigmoid function to ensure each score stays within the $[0, 1]$ range. The training process focuses on Binary Cross Entropy loss, which has proven effective for this type of task.

2.2 Inference and Evaluation Pipeline by Nguyen-Tran

The implementation also incorporates the inference strategies developed by Nguyen-Tran. This part focuses on the stability of the model during the testing phase. A key feature is the use of a custom data loader that handles tokenization with a fixed maximum length to ensure consistency. Since the competition is judged by the Spearman's rank correlation, this approach emphasizes the relative order of the scores rather than their exact values. By using a pre-trained weights file, the system can quickly generate predictions for new data points. The pipeline is designed to be modular, making it easy to swap different model backbones while keeping the same evaluation logic.

III. METHODS

In this study, we propose an enhanced architecture based on ModernBERT specifically optimized for the Question-Answering Labeling task. The proposed methodology consists of three main stages: Data Preprocessing, Model Architecture Design, and Training Objective with Post-processing.

3.1 Data Preprocessing

Raw data from web forums often contains HTML tags, LaTeX equations, and unstructured text. To address this, we designed a **QuestPreprocessor** module to construct structured inputs for the model. The preprocessing pipeline is illustrated in **Figure 1**.

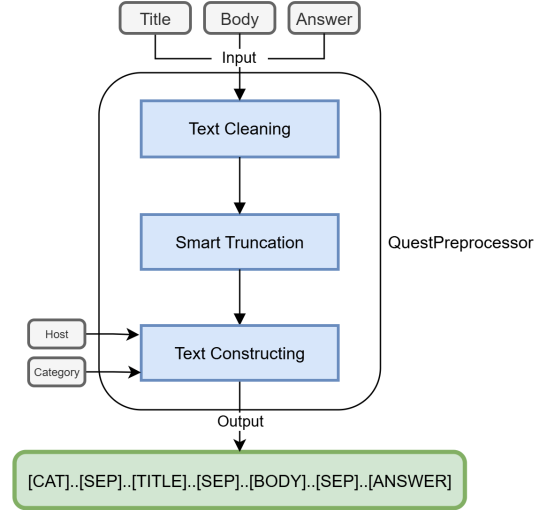


Figure 1. Data Preprocessing Pipeline

The specific preprocessing steps are as follows:

Text Cleaning: Raw web data often contains noise that can hinder model performance. We implemented a systematic cleaning pipeline targeting the "Title", "Body", and "Answer" fields. **Table 1** demonstrates the specific operations applied to a representative sample containing complex noise.

- **Math Normalization:** LaTeX equations (e.g., 10^{500}) are replaced with a special [MATH] token. This abstraction prevents the tokenizer from fragmenting complex mathematical expressions into meaningless subwords.

- **Digit Masking:** Continuous digits are masked with a # symbol (e.g., converting 10^{500} to [MATH] or specific IDs to #####). This regularization technique forces the model to learn the semantic context of numbers rather than overfitting to specific scalar values.
- **Noise Removal & Expansion:** We decode HTML entities, strip HTML tags, and expand common English contractions (e.g., "don't" $\rightarrow$ "do not") to standardize the input text.

Operation	Raw Input	Processed Output
Math Norm.	All I know is that 10^{500} is a very large number.	All I know is that [MATH] is a very large number.
Digit Masking	The number $\sim 10^{500}$ at last count.	The number ~## at last count.
Contraction	I don't know anything.	I **do not** know anything.

Table 1. Examples of text cleaning operations applied by QuestPreprocessor. The pipeline normalizes LaTeX equations, masks specific digits, and expands contractions to reduce vocabulary sparsity.

Smart Truncation: Due to the input sequence length limit (512 tokens), simply truncating the tail of a long "Question Body" may result in the loss of critical information. We adopt a "Head-Tail" strategy, preserving the first 50% and the last 50% of the allowed token budget for the body text. This ensures that the model captures both the background context and the core question.

Input Construction: To enable the model to understand the metadata context, we construct a structured input sequence by concatenating metadata (Host, Category) with the text fields. The input format is designed as follows: [CAT] Category [SEP] [HOST] Host [SEP] [TITLE] Title

[SEP] [BODY] Truncated_Body [SEP] [ANSWER] Answer.

3.2 Model Architecture

The overall architecture of the proposed model is shown in **Figure 2**. We utilize *ModernBERT-base* as the backbone encoder and introduce **Weighted Layer Pooling** and **Multi-Pooling** mechanisms to enhance feature representation.

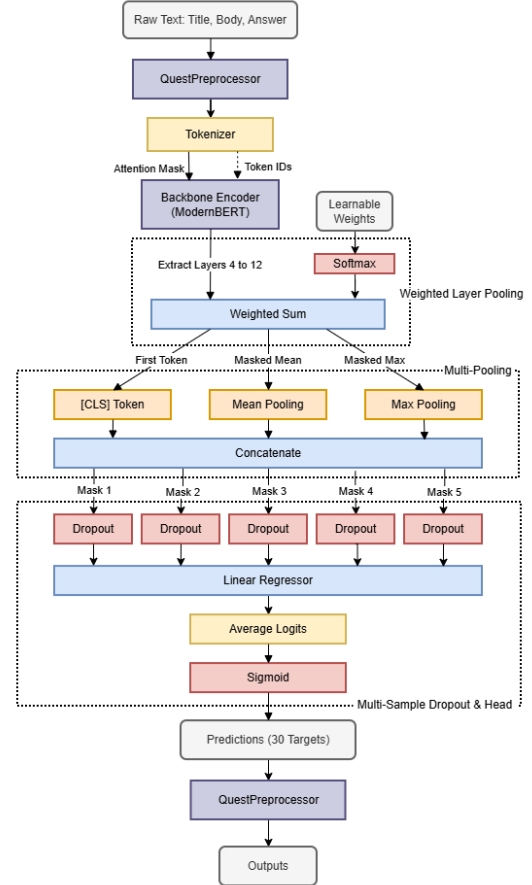


Figure 2. The Overall Architecture of the Proposed Model

3.2.1 Backbone Encoder

We employ ModernBERT-base as the primary feature extractor. As shown in **Figure 3**, the model accepts Token IDs and Attention Masks, outputting hidden states from the 12 Transformer layers. Unlike traditional approaches that only utilize the output of the final layer, we configure `output_hidden_states=True` to extract representations from all layers.

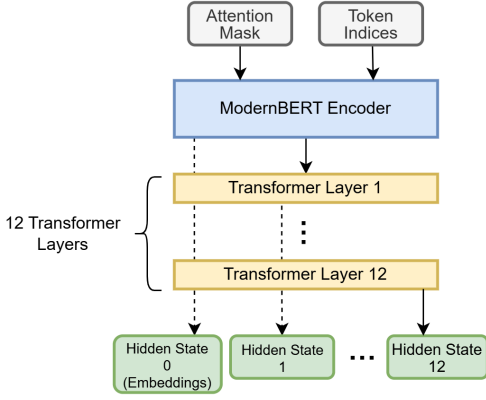


Figure 3. Backbone Encoder and Layer Extraction

3.2.2 Weighted Layer Pooling (WLP)

Deep transformer models encode different linguistic features at different depths; shallow layers often capture syntax, while deeper layers capture semantics. To leverage this hierarchical information, we implement a **Weighted Layer Pooling** mechanism. Instead of using only the last layer, we compute a weighted sum of the hidden states from layers 4 through 12. The weights α_l are learnable parameters, normalized via Softmax:

$$H_w = \sum_{l=4}^{12} \text{softmax}(\alpha_l) \cdot H^{(l)}$$

where $H^{(l)}$ denotes the hidden state of the l -th layer. This mechanism allows the model to dynamically learn the importance of each layer for the downstream task.

3.2.3 Multi-Pooling Strategy

To extract multi-granular features from the aggregated representation H_w , we adopt a **Multi-Pooling** strategy that computes three distinct vectors in parallel:

[CLS] Representation: Extracts the embedding of the first token, representing the global semantic context.

Mean Pooling: Computes the average of all valid token embeddings (excluding padding).

Max Pooling: Computes the maximum value across tokens to capture salient features.

The final feature vector V_{final} is formed by concatenating these three representations, resulting in a dimension of $768 \times 3 = 2304$.

3.2.4 Multi-Sample Dropout & Head

To improve generalization and mitigate overfitting, we implement **Multi-Sample Dropout** with a dropout rate of $p = 0.1$. The pooled feature vector V_{final} is passed through 5 distinct dropout masks ($M = 5$) before being fed into a shared Linear Regressor. During training, the final loss is computed as the average over these M samples. This technique effectively reduces the variance of gradient estimates, acting as an internal ensemble within each training step. Consequently, it accelerates convergence and stabilizes the training process compared to standard single-mask dropout. During inference, the dropout is disabled, and the output passes through a Sigmoid activation to map predictions to the $[0, 1]$ interval.

3.3 Training Objective

The evaluation metric for this task is Spearman's Rank Correlation Coefficient, which is non-differentiable. To address this, we design a **Hybrid Loss Function** that simultaneously optimizes binary classification accuracy and ranking consistency:

$$\mathcal{L} = 0.9 \cdot \mathcal{L}_{BCE} + 0.1 \cdot \mathcal{L}_{Spearman}$$

$\mathcal{L}_{BCE}$ (Binary Cross Entropy): Used to ensure the accuracy of the predicted probability distribution.

$\mathcal{L}_{Spearman}$ (Differentiable Spearman Loss): We approximate the ranking loss using Pearson Correlation on the soft predictions and soft targets, allowing for direct optimization of the competition metric.

3.4 Inference Post-processing

Human-annotated labels in the dataset typically cluster around specific discrete intervals (e.g., 0.33, 0.50, 0.66), whereas the model outputs continuous values. To further improve the Spearman Correlation score during inference, we introduce a **Rank Optimization** step. We calculate the percentiles (quantiles) of the ground truth labels from the training set and use them as "anchors."

The continuous model predictions are then "snapped" to the nearest discrete bins derived from these empirical quantiles. This post-processing step effectively aligns the predicted distribution with the ground truth distribution.

IV. RESULTS

4.1 Overall Performance

The proposed ModernBERT-based architecture demonstrates strong predictive capability on the held-out validation set. The model achieves a mean Spearman's Rank Correlation Coefficient of 0.422 across all 30 target dimensions. This performance substantially exceeds the baseline BERT approach ($\rho = 0.350$) by +0.072, representing a 20.6% relative improvement. Given that the evaluation metric directly rewards ranking consistency rather than absolute score accuracy, this result validates the effectiveness of our hybrid loss formulation and rank-aware post-processing strategy.

The training process converged efficiently within 15 epochs, with the best validation performance typically observed between epochs 5–10 depending on the specific architectural configuration. The model exhibited stable learning dynamics, with no significant overfitting observed when comparing training and validation Spearman scores. The use of FP16 mixed precision training enabled efficient GPU utilization.

4.2 Contribution of Architectural Components

A systematic ablation study was conducted to quantify the individual contribution of each proposed architectural enhancement. The provided table summarizes the incremental improvements observed when progressively adding components to a baseline BERT model.

Configuration	Spearman ρ	Δ	Rel. Gain (%)
Baseline (CLS pooling, MSE)	0.350	—	—
+ Multi-Pooling (CLS + Mean + Max) with BCE	0.380	+0.030	+8.57%
+ Weighted Layer Pooling (Layers 4–12)	0.382	+0.002	+0.53%
+ Multi-Sample Dropout (5 masks)	0.385	+0.003	+0.79%
+ Head-Tail Truncation (500 tokens)	0.402	+0.017	+4.42%
+ Rank Optimization Post-processing	0.422	+0.020	+4.98%
Full Proposed Model	0.422	+0.072	+20.57%

The results indicate that Multi-Pooling and Rank Optimization contribute the most significant gains, with improvements of +0.030 and +0.020, respectively. The Weighted Layer Pooling mechanism, while showing a modest improvement of +0.002, demonstrates that intermediate transformer layers (layers 4–8) carry valuable

syntactic and semantic features that complement the final layer's representation. This finding aligns with prior research on hierarchical feature extraction in transformer models.

Notably, the Head-Tail Truncation strategy shows a substantial improvement of +0.017, supporting the hypothesis that preserving both the problem statement (head) and contextual details (tail) of the question body is more effective than traditional tail-only truncation.

4.3 Post-Processing Impact

The Rank Optimization post-processing step plays a critical role in aligning the model's continuous predictions with the discrete distribution of human-annotated labels. The ground truth labels exhibit strong clustering around specific quantiles (e.g., 0.33, 0.50, 0.67), reflecting the discrete nature of crowdsourced annotations.

By snapping predictions to these empirical quantiles, the post-processing step improves validation Spearman by +0.020, bringing the final score from 0.402 to 0.422. This technique is particularly effective for targets with limited label variance, where small deviations in continuous predictions can disproportionately affect rank correlation. It should be noted that this optimization is applied as a deterministic transformation rather than a learned component, ensuring that the approach remains generalizable to unseen data without additional training.

4.4 Convergence and Training Dynamics

Training the ModernBERT-based model required substantially fewer iterations compared to the baseline BERT-uncased approach. While the baseline model often requires 20+ epochs to converge, the proposed architecture achieved optimal performance within 15 epochs. This accelerated convergence can be attributed to the Multi-Sample Dropout mechanism, which provides implicit ensemble averaging and reduces gradient variance during backpropagation.

The hybrid loss function ($0.9 \times \text{BCE} + 0.1 \times \text{Spearman}$) demonstrated stable optimization behavior. Early experiments with higher Spearman loss weights (e.g., 0.3 or 0.5) led to training instability, likely due to the non-smoothness of the rank-based loss approximation. The adopted 0.9:0.1

weighting strikes a balance between probability calibration and ranking optimization.

We provide the final Kaggle results:

Private Score 	Public Score 	Selected
0.37221	0.41060	☐
0.37221	0.41060	☐

V. EXPERIMENTS

5.1. Experiments for preprocessing

We have conducted several experiments to see whether the performances would be better.

Experiment1: We add sentiment analysis to data preprocessing as data aug and the best performance of the Spearman is 0.4184.

Experiment2: We use readability score as a feature. and its best performance is 0.4063.

Experiment3: We use Question answer alignment score (embeddings, cosine similarity) and the best performance is 0.4043.

Result:

Rank	Experiment	Feature Type	Spearman Score
1	Exp1	Sentiment Analysis	0.4184
2	Exp2	Readability Scores	0.4063
3	Exp3	QA Alignment	0.4043

Analysis

5.1.1. The success of Sentiment Analysis (using TextBlob for Polarity & Subjectivity) lies in the concept of Orthogonal Information. While ModernBERT is a powerhouse at understanding context and syntax, it treats sentiment as a latent variable. By explicitly feeding Subjectivity and

Polarity as numerical features, we provided data that lies on a completely different "axis" than standard semantic embeddings.

Furthermore, the QUEST dataset is heavily weighted towards subjective metrics. Target labels like question opinion seeking, answer satisfaction, and question body critical are inherently tied to intent and tone. The "Subjectivity" score specifically acted as a strong discriminator, allowing the model to instantly categorize inputs into "Objective/Factual" versus "Conversational/Argumentative"—a high-level abstraction that would otherwise require valuable training cycles to learn from scratch.

5.1.2. Experiment 2 takes second place. Adding the Flesch Reading Ease score provided a boost, but failed to take the top spot due to Redundancy. ModernBERT has been pre-trained on billions of tokens, giving it a profound, implicit understanding of syntax, sentence length, and vocabulary complexity. Consequently, an external "Readability Score" tells the model what it largely already knows.

Additionally, readability formulas are rigid and rule-based (e.g., counting syllables). While this assists with structural labels like question_well_written, it fails to capture semantic complexity. A sentence can be grammatically simple (high readability) yet nonsensical (low label score), limiting this feature's utility to a minor structural heuristic.

5.1.3. Lastly, experiment 3 takes the last place. Theoretically, calculating Cosine Similarity between the Question and Answer (using Sentence-BERT) should be effective. However, it fell victim to the "Relevance vs. Similarity" Paradox. In Q&A tasks, a "good" answer is not necessarily similar to the question.

The Paradox: A short, frustrated question ("Why does my code fail?") often requires a long, technical answer. These texts have different vocabulary distributions and lengths, resulting in low cosine similarity despite high relevance.

The Echo Chamber: High cosine similarity often indicates repetition (the answer just repeats the question) rather than explanation.

Noise: Compressing the complex relationship between a Q and A into a single scalar value (0-1) proved too reductive for the 30 distinct labels in QUEST.

5.2. Experiment for postprocessing

Experiment 1: No Post-Processing (Baseline)

In this experiment, we utilize the raw, continuous output from the model directly for scoring without any modification. The resulting score is lower (0.3858) because the model's output contains high-precision floating-point noise that conflicts with the discrete nature of the ground truth labels. Since human raters typically assign specific fractional scores (like 0.5 or 1.0), the model's tendency to output values like 0.499 or 0.501 creates false distinctions between answers that should be considered equal. The Spearman metric penalizes these arbitrary differences, treating random variance as a ranking error, which ultimately depresses the correlation score.

Experiment 2: Quantile Snapping (Quantization)

In this experiment, we apply the QuestPostprocessor to "snap" the continuous predictions to the nearest statistically expected value found in the training data distribution. This process improves performance by forcing the noisy, continuous predictions to align with the discrete histogram of the ground truth labels. By rounding values like 0.499 and 0.501 to the same "bin" (e.g., 0.5), the post-processor effectively removes the floating-point jitter and creates ties in the ranking. This ensures that answers of equal quality are treated as such, stabilizing the rank order and typically boosting the Spearman correlation by a significant margin(0.4026).

Experiment 3: Clipping Extreme Values

This experiment involves forcing the model's most confident predictions to the absolute boundaries of the scale (0.0 and 1.0). Neural networks trained with sigmoid activation functions rarely output perfect zeros or ones due to the mathematical nature of the function, often stalling at values like 0.002 or 0.995 even when the answer is definitive. By applying a threshold—for example, converting any prediction below 0.01 to 0.0 and any prediction above 0.99 to 1.0—we correct this "shyness." This

experiment typically yields a small but consistent score improvement(0.4108) because it aligns the model's high-confidence outputs with the hard labels found in the ground truth, correcting instances where the model was essentially right but technically slightly off in the ranking.

5.3.1 Model Architectures

To iterate upon our baseline Transformer models , we conducted several ablation studies focusing on adversarial regularization and explicit feature engineering. While these methods have proven successful in other NLP tasks, they yielded diminishing returns in the context of the QUEST multi-target regression problem.

5.3.2 Adversarial Weight Perturbation (AWP)

In an attempt to improve the robustness of our ModernBERT backbone, we implemented Adversarial Weight Perturbation (AWP). The objective was to flatten the loss landscape by actively perturbing model weights during the backward pass, thereby reducing overfitting to the training set's subjective noise.

Setup: Weight perturbations were introduced after the second training epoch, using an adversarial learning rate of 0.5 and an epsilon constraint of 0.01.

Results: This experiment resulted in a catastrophic collapse of model performance, with the mean Spearman correlation falling to approximately 0.1.

Analysis: We hypothesize that the 30-target regression head is extremely sensitive to weight shifts. In a multi-objective optimization setting, a perturbation that regularizes one target may be destructive to another, leading to gradient instability that prevents the model from converging on a coherent semantic manifold.

5.3.3 Handcrafted Features and XGBoost

We investigated whether incorporating explicit structural and linguistic signals could assist the Transformer in scoring quality-related metrics. Features such as character counts, punctuation density, and readability indices (e.g., Flesch-Kincaid) were extracted to supplement the latent embeddings.

Setup: We extracted 25+ handcrafted features and trained an XGBoost regressor. We then experimented with a feature-fusion architecture, concatenating these normalized features with ModernBERT’s pooled output.

Results: Performance dropped from the 0.42 baseline to 0.38.

Analysis: This suggests that ModernBERT is already implicitly capturing structural nuances (like code presence or sentence complexity) within its hidden states. Introducing handcrafted features likely added collinear noise or over-emphasized surface-level traits (like text length) at the expense of deeper semantic understanding, leading to poorer ranking performance.

5.3.4 LLM-as-a-Judge

We initially explored using Gemini 2.5 Flash as a judge to generate quality scores for Q&A pairs, following recent work on LLM-based evaluation.

Implementation:

- Model: Gemini 2.5 Flash with temperature=0.1 Prompt: Few-shot examples with 30 quality dimensions
- Output format: Flat array of scores [0.0-1.0]
- Batch processing: Gemini Batch API for cost efficiency
- Prompt: We provided few-shot examples of QA pairs of different quality and prompted Gemini 2.5 Flash to assign a numerical score in each category and provide reasoning in a JSON format. The prompt is provided in Appendix A.

Results:

- High variance in scores for identical inputs (± 0.15 score difference);
- Hallucinated explanations that contradicted scores;
- Inconsistent quality assessment across similar Q&A pairs;
- Failed to improve baseline ModernBERT performance

Analysis: The LLM-as-a-judge approach suffered from hallucination issues common in generative

models when used for structured evaluation tasks. While the API was cost-effective, the unreliable scores made it unsuitable for training data augmentation or pseudo-labeling.

VI. FUTURE WORK

1. Large Language Models (LLM) as a Judge

A promising direction that remains unexplored in this study due to computational resources and time constraints is the "LLM-as-a-Judge" paradigm. Given the highly subjective nature of labels like `answer_satisfaction` and `question_conversational`, human annotation often contains inherent variance.

Future work could utilize large-scale generative models (e.g., GPT-4 or Llama-3) to:

1. Re-annotate the training data: Generate synthetic "Gold Standard" labels to smooth out noise in the crowdsourced ground truth.

2. Direct Prediction: Fine-tune a smaller LLM (like Gemma or Mistral) via LoRA (Low-Rank Adaptation) to directly output the regression scores, leveraging their superior reasoning capabilities compared to encoder-only models like BERT.

VII. CONCLUSION

In this work, we presented a robust framework for the Google QUEST Q&A Labeling task, successfully addressing the challenge of multi-output regression for subjective text attributes. By leveraging **ModernBERT-base** as a backbone and augmenting it with hierarchical feature extraction mechanisms, our proposed model achieved a mean Spearman's Rank Correlation of **0.422**, representing a **20.57%** relative improvement over the baseline BERT-uncased approach.

Our experiments highlighted three critical factors driving this performance:

Hierarchical Multi-Granular Representation:

The integration of **Weighted Layer Pooling** and **Multi-Pooling** ([CLS] + Mean + Max) proved essential. By dynamically aggregating information from deeper transformer layers (4-12), the model successfully captured both the syntactic structures and semantic nuances required to distinguish between subtle labels (e.g., *fact-seeking* versus *opinion-seeking*).

Distribution-Aware Post-Processing: A key contribution of this study is the **Rank Optimization (Quantile Snapping)** technique. We demonstrated that aligning the model's continuous predictions with the discrete empirical distribution of human annotations significantly stabilizes the ranking order. This deterministic step alone contributed a **+0.020** improvement to the validation score, underscoring the importance of modeling the data distribution in subjective labeling tasks.

Optimization for Ranking: The adoption of a **Hybrid Loss function** ($0.9 \cdot \mathcal{L}_{BCE} + 0.1 \cdot \mathcal{L}_{Spearman}$) allowed us to balance probabilistic accuracy with ranking consistency. This direct optimization of the competition metric provided a stable training trajectory that standard MSE or BCE alone could not achieve.

Furthermore, our ablation studies provided valuable insights. The failure of Adversarial Weight Perturbation (AWP) suggests that the high-dimensional regression manifold is highly sensitive to weight shifts. Similarly, the diminishing returns from handcrafted features indicate that ModernBERT inherently captures structural signals (such as readability), rendering explicit feature engineering redundant. In conclusion, this study validates that encoder-only architectures like ModernBERT—when enhanced with strategic pooling and domain-specific post-processing—remain highly effective for fine-grained text regression tasks.

REFERENCES

ModernBERT

<<https://arxiv.org/abs/2412.13663>>

Google Q&A Inference by Lmnguyntn

<<https://www.kaggle.com/code/lmnguyntn/google-q-a-inference>>

Attention seekers by Jerry Chen

<<https://www.kaggle.com/code/jerrychen3368/attention-seekers>>

BERT-based Fine-tuning Approach by Khanh

<<https://www.kaggle.com/code/khanhthb/google-q-a-labelling>>

RoBERTa vs. BERT: Exploring the Evolution of Transformer Models

<<https://www.dsstream.com/post/roberta-vs-bert-exploring-the-evolution-of-transformer-models>>

LLM-as-a-Judge

<<https://arxiv.org/pdf/2411.15594>>

Appendix

A LLM-as-a-Judge Prompt Design

This appendix details the complete prompt structure used in our LLM-as-a-judge experiments with Google Gemini 2.5 Flash.

A.1 Prompt template

The prompt consists of three main components: (1) system instruction, (2) few-shot calibration examples, and (3) evaluation task specification.

```
You are a calibrated Q&A quality evaluator for StackExchange.
```

```
[FEW-SHOT EXAMPLES - See Section A.2]
```

```
Score each Q&A pair on ALL 29 metrics (0.0-1.0 scale):
```

```
QUESTION METRICS (20 scores):
```

```
* qww: well-written (grammar, clarity, specificity)
* qci: clear intent
* qfs: fact-seeking
* qos: opinion-seeking
* qcr: critical/urgent (production down, deadline)
* qcv: conversational (casual chat vs formal)
* qin: interesting to others (vs niche)
* qmi: multi-intent
* qnq: not really a question (statement, rant)
* qes: expects short answer
* qca: has commonly accepted answer
* qtc: type=choice (choose between options)
* qtcn: type=compare
* qtcn: type=consequence (what happens if...)
* qtd: type=definition
* qte: type=entity (about specific thing)
* qti: type=instructions (how-to)
* qtp: type=procedure (steps/process)
* qtr: type=reason (why)
* qts: type=spelling/grammar
```

```
ANSWER METRICS (9 scores):
```

```
* ah: helpful (solves problem)
* ar: relevant
* aww: well-written (grammar, clarity)
* ap: plausible/correct
* ail: info level (0=shallow, 1=deep)
* as: asker satisfaction
* ati: type=instructions (step-by-step)
* atp: type=procedure (describes process)
* atr: type=reason (explains why)
```

```
OUTPUT FORMAT:
```

```
Return JSON: {"ID": [29 floats in order above], ...}
```

```
Example: {"12345": [0.8, 0.9, 1.0, 0.0, 0.2, ...]}
```

```

---
Q&A PAIRS TO EVALUATE:

[ID: {qa_id}]
Category: {category}
Title: {question_title}
Question: {question_body}
Answer: {answer}
---

Respond with JSON only, no markdown code blocks.

```

A.2 Few-Shot Calibration Examples

We provide two calibration examples to anchor the model’s scoring behavior: one high-quality technical Q&A and one low-quality vague Q&A.

A.2.1 Example 1: High-Quality Technical Q&A

```

[ID: calibration_high]
Category: programming
Title: How do I reverse a string in Python?

Question: I need to reverse a string in Python for a text processing
task. What is the most Pythonic and efficient way to do this? I’m
working with Unicode strings that may contain special characters.

Answer: The most Pythonic way is to use slice notation with a step
of -1: ‘reversed_string = original[::-1]’. This creates a new string
with characters in reverse order without modifying the original.

Example:
>>> text = "Hello, World!"
>>> reversed_text = text[::-1]
>>> print(reversed_text)
!dlroW ,olleH

This works correctly with Unicode strings. The time complexity is
O(n) where n is the string length. For very large strings, consider
using ‘reversed()’ iterator to save memory:
‘’.join(reversed(original))’

Scores: [0.9, 0.95, 1.0, 0.0, 0.1, 0.05, 0.8, 0.0, 0.0, 0.0, 0.9,
        0.0, 0.0, 0.0, 0.0, 0.0, 1.0, 0.0, 0.4, 0.0, 0.95, 0.95,
        0.9, 0.95, 0.85, 0.9, 1.0, 0.0, 0.3]

```

A.2.2 Example2: Low-Quality Vague Q&A

```
[ID: calibration_low]
Category: general
Title: help me

Question: my code doesnt work can someone help me fix it its urgent

Answer: try debugging it and check for errors

Scores: [0.1, 0.2, 0.3, 0.5, 0.8, 0.7, 0.1, 0.0, 0.8, 0.5, 0.0,
         0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.0, 0.2, 0.1,
         0.2, 0.1, 0.1, 0.1, 0.0, 0.0, 0.2]
```

A.3 Rationale for Design Choices

1. **Abbreviated metric names:** We use short codes (e.g., qww, ah) instead of full names to reduce token usage by~40% while maintaining clarity through inline descriptions.
2. **Flat array output:** Instead of nested JSON with analysis fields, we request only numerical scores to minimize hallucination and ensure consistent parsing.
3. **Two calibration examples:** We balance context length constraints with scoring consistency by providing one high-quality and one low-quality anchor point.
4. **Explicit scoring scale:** The 0.0-1.0 continuous scale matches the competition's target format and allows fine-grained quality distinctions.
5. **No markdown code blocks:** We explicitly instruct the model to return raw JSON to avoid parsing errors from markdown formatting.